```
searchNode()        {
    if( root == NULL)                          O(1)
        return NULL;                           O(1)
    if (root -> filename == fname)             O(L)
        return root;                           O(1)
    if (fname < root -> filename)              O(L)
        return searchNode(...)                 T(n-1)
    else
        return searchNode(...)                 T(n-1)
}
```

$\Longrightarrow$ $2 \times O(L)$

AVL Height = $\log(U)$

Total = $2L \times \log(U) = O(L \log U)$

$= O(\log U)$ ∵ L const per run

```
printResult()
    for (int i(0); i< result->paths.size(); i++)      K iterations
        cout << result -> paths[i]                    1 per iter
```

$\Longrightarrow$ Iterations = K

Per itr = 1 print + 1 inc + 1 comp = 3 ops

Total = 3K ops = $O(K)$

$\Longrightarrow$ Full Search = searchNode + printResult

$= O(L \log U) + O(K)$

$= O(\log U + K)$ ←

avl_insert()

if (avlNode == nullptr) $O(1)$

if (fname == avlNode→filename) $O(L)$

else if (fname < avlNode→filename) $O(L)$

(recursive left or right)

Per level: $2L$ ops

Levels: $\log(U)$

total: $2L \times \log(U) = O(\log U)$

→ updateheight() on way back up:

n→height = 1 + max(left, right) 5 ops

total = $5 \times \log(U) = O(\log U)$

→ getBalance() + rotation check

getBalance() 2 gethelght() + 1 sub = 3 ops

4 if-checks for LL - RR 4 ops

At most 1 rot:

rotate left or right 4 pointers assigns + 2 Update heights

4 + 10 = 14 ops

total = $O(U)$ per level × $\log(U)$ levels = $O(\log U)$

⟹ Total insert = $O(\log U) + O(\log U) + O(\log U)$

= $O(\log U)$ ←

loadDirectory()

for(auto& entry : directory_iterator)

Per entry (file)

is_regular_file()	$O(1)$
.file name().string()	$O(L)$
fs::absolute()	$O(L)$
index.insert()	$O(\log u)$ — if unique
	$O(1)$ — duplicate

Per entry(directory):

is_directory()	$O(1)$
loadDirectory()	recurring deep

→ Files = n

directories = d

Total itr = $n + d$

Total insert TC = $u \cdot O(\log u) + (n-u)\, O(1)$

$= O(u \log u) + O(n-u)$

⇒ Total loadDirectory:

$= O(n+d) + O(u \log u) + O(n-u)$

$= O(n \log u)$ ←

```
freeTree()
    if (root)                          {          O(1)
        freeTree(root->left)                      securse
        freeTree(root->right)                     securse
        delete root                               O(1)
    }
```

→ Nodes = U | Per node = 4ops

Total = 4U ops = O(U)

```
refresh()                              {
    freeTree(root)                                O(U)
    root = NULLptr                                O(1)
    loadDirectory(rootpath, *this)                O(n log U)
}
```

⟹ Total = O(U) + O(1) + O(n log U)

Total = O(n log U) ←